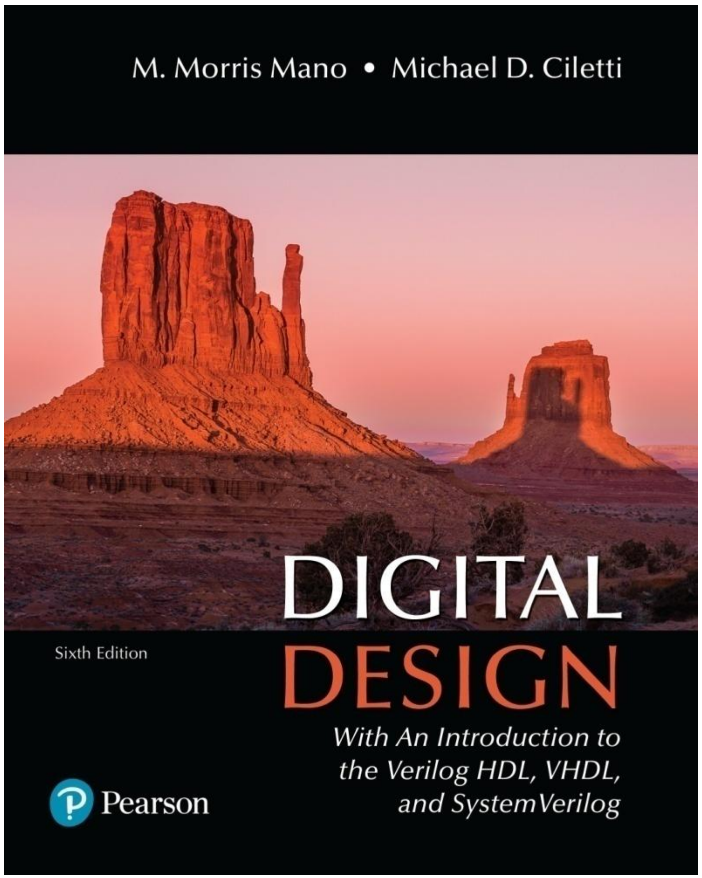
Digital Design

With an Introduction to the Verilog HDL, VHDL, and SystemVerilog

6th Edition



Chapter 08

Design at the Register Transfer Level

Table 8.1
Verilog 2001, 2005 HDL Operators.

Operator Type	Symbol	Operation Performed	Priority Group
Arithmetic	+	addition	1 (unary), 4 (binary)
	–	subtraction	1 (unary), 4 (binary)
	*	multiplication	3
	/	division	3
	**	exponentiation	2
	%	modulus	2
Bitwise or Reduction	~	negation (complement)	1
	&, ~&	AND, NAND (reduction)	1
	|,~|	OR, NOR (reduction)	1
	^, ~^	XOR, XNOR (reduction)	1
	^, ~^, ^~	XOR, XNOR (binary)	9
Logical	!	negation	1
	&&	AND (binary)	11
	||	OR (binary)	12
	&	AND (binary)	8
	|	OR (binary)	10
Shift	>>	logical right shift	5
	<<	logical left shift	5
	>>>	arithmetic right shift	5
	<<<	arithmetic left shift	5
Relational	>	greater than	6
	<	less than	6
	<=	less than or equal	6
	>=	greater than or equal	6
Equality	==	equality	7
	!=	inequality	7
	===	case equality	7
	!==	case inequality	7
Conditional	? :	ternary selection	13
Concatenation	{ } {{}}	joins operands	14

Table 8.2
Verilog Operator Precedence.

<table>
<tr><td>+ − ! ~ & ~& | ~| ^ ~^ ^~ (unary)</td><td>Highest precedence</td></tr>
<tr><td>**</td><td></td></tr>
<tr><td>* / %</td><td></td></tr>
<tr><td>+ − (binary)</td><td></td></tr>
<tr><td>≪ ≫ <<< >>></td><td></td></tr>
<tr><td>< <= > >=</td><td></td></tr>
<tr><td>== != === !==</td><td></td></tr>
<tr><td>& (binary) (Bitwise, Reduction AND)</td><td></td></tr>
<tr><td>^ ^~ ~^ (binary) (Bitwise, Reduction OR)</td><td></td></tr>
<tr><td>| (binary)</td><td></td></tr>
<tr><td>&& (logical AND)</td><td></td></tr>
<tr><td>|| (logical OR)</td><td></td></tr>
<tr><td>?: (conditional operator)</td><td></td></tr>
<tr><td>{ } { { } } (concatenation)</td><td>Lowest precedence</td></tr>
</table>

Table Operations

Operation		Result	Operation
A and B		0000	Bitwise AND
A or B		1010	Bitwise OR
not A	-- Logical negation		
$A > B$	-- Greater than	TRUE	Relational greater than
$A = B$	-- Equals	FALSE	Relational identity

Pearson

Figure PE 8.8

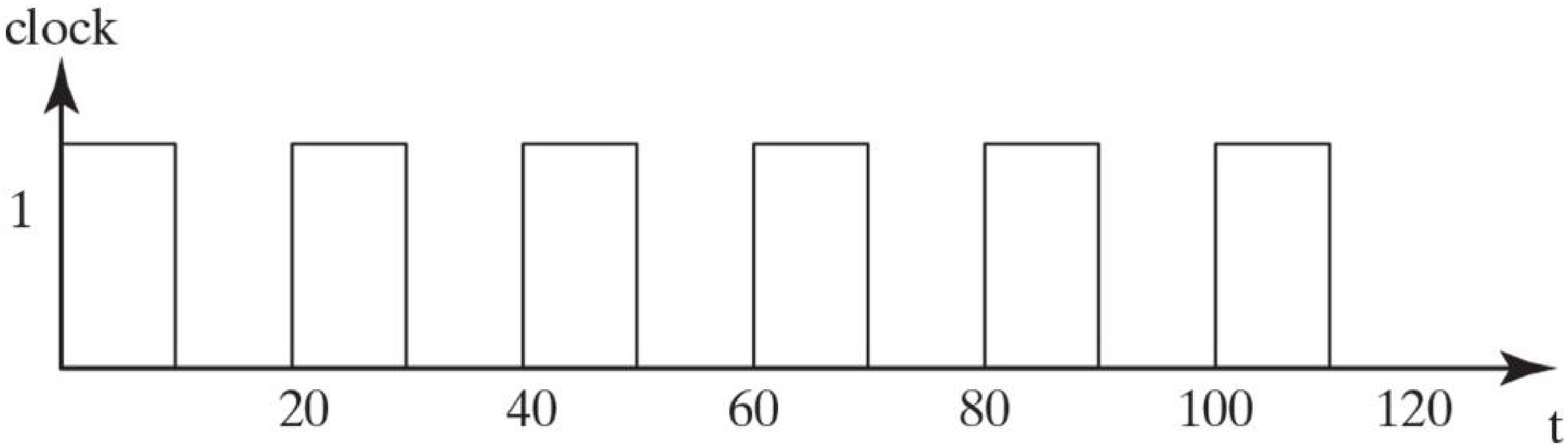


Figure PE 8.9

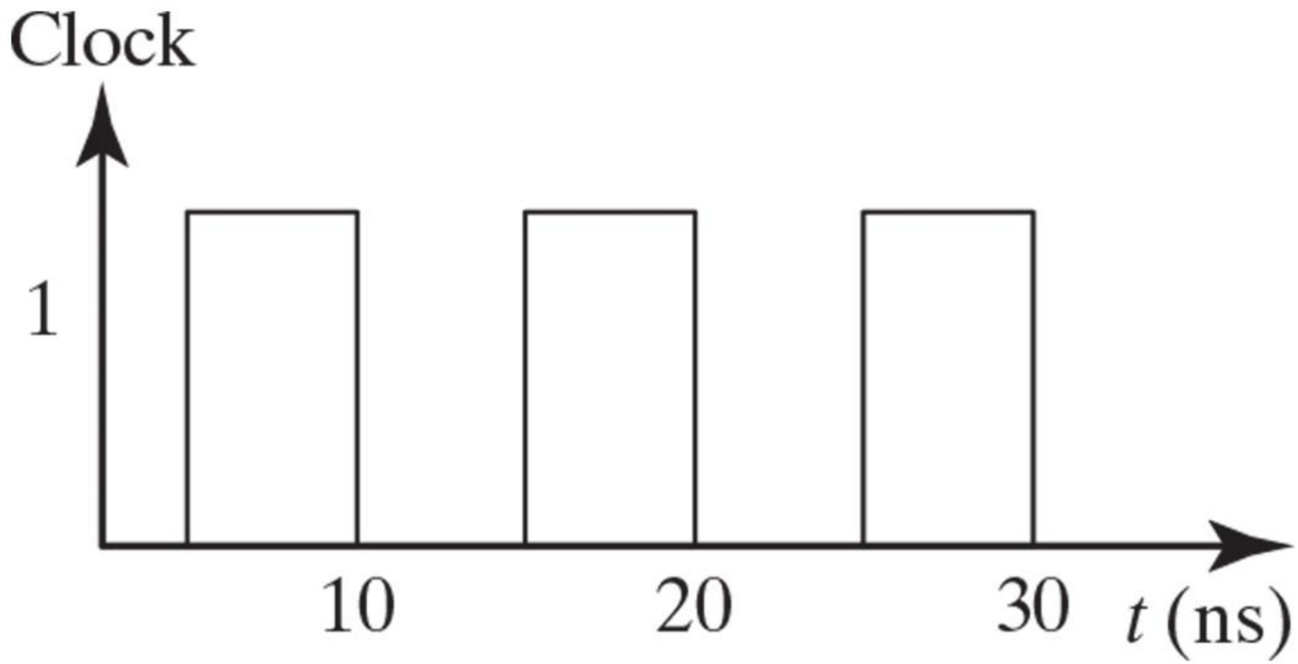


Figure 8.1
A simplified flowchart for HDL-based modeling, verification, and synthesis.

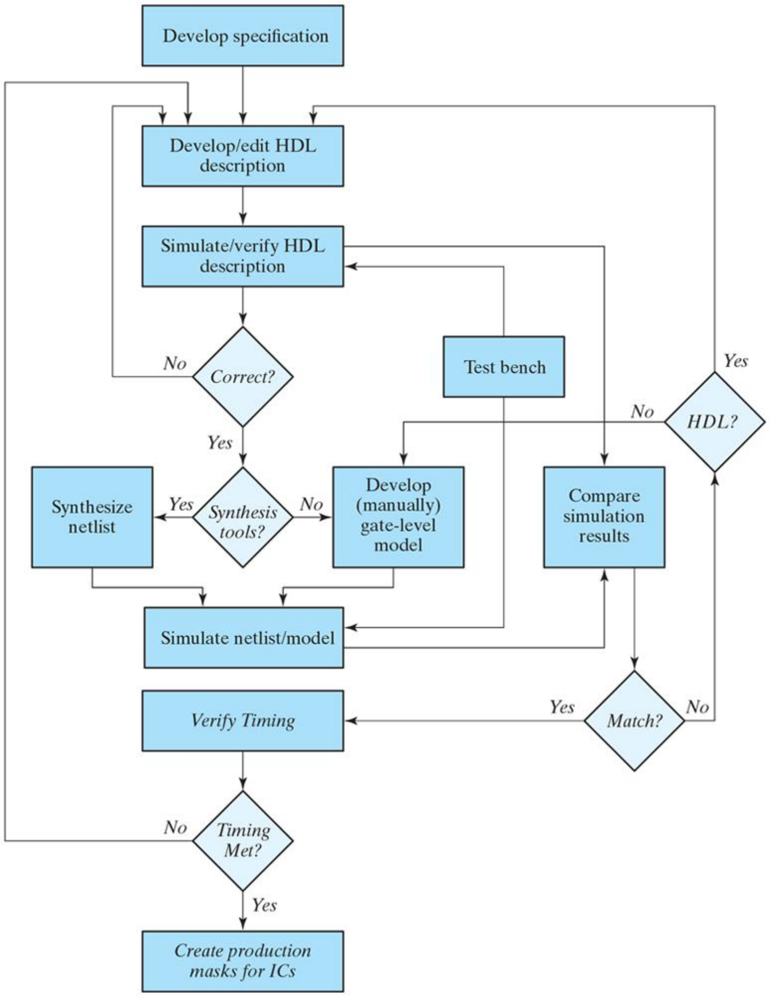


Figure 8.2
Control and datapath interaction.

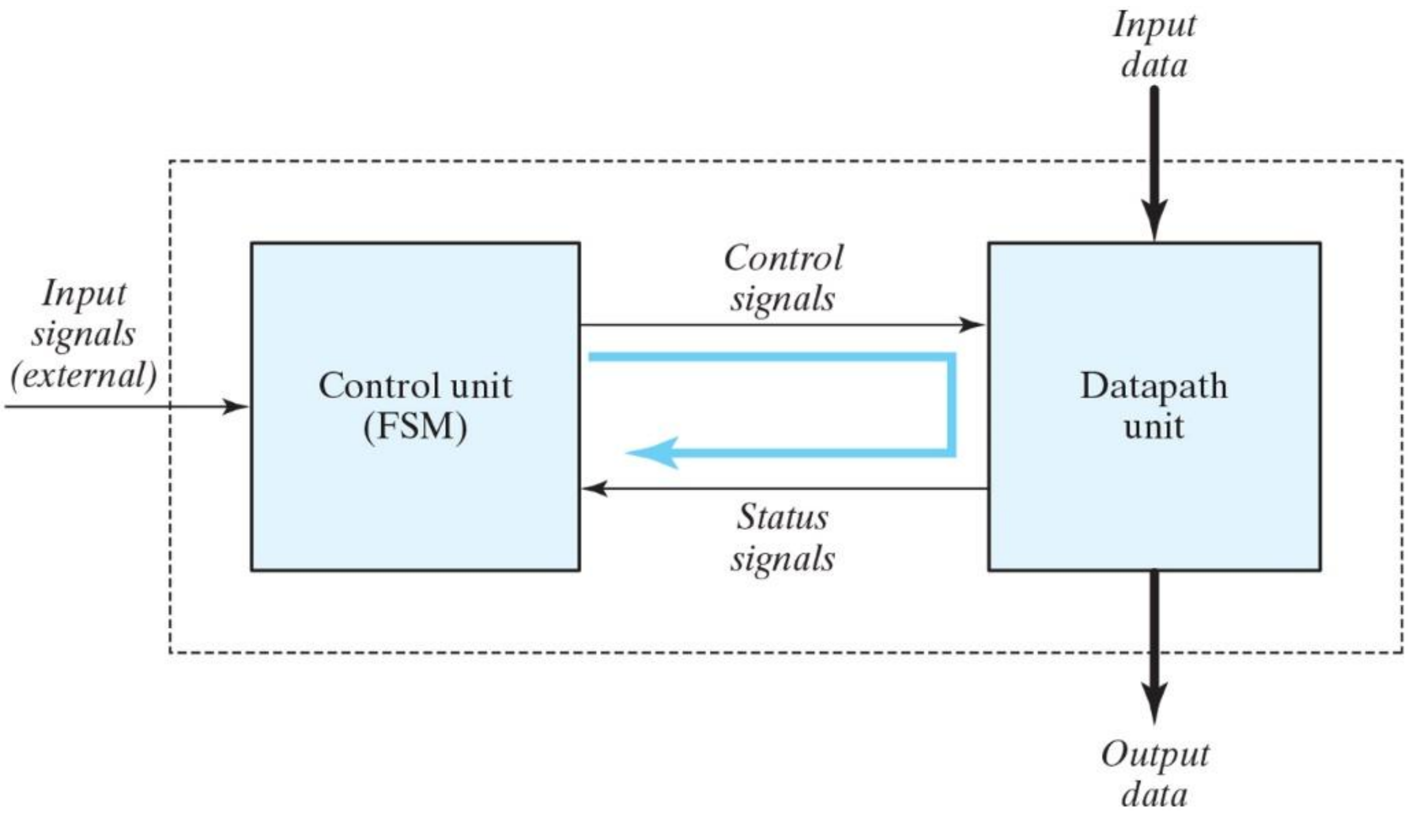


Figure 8.3
ASM chart state box.

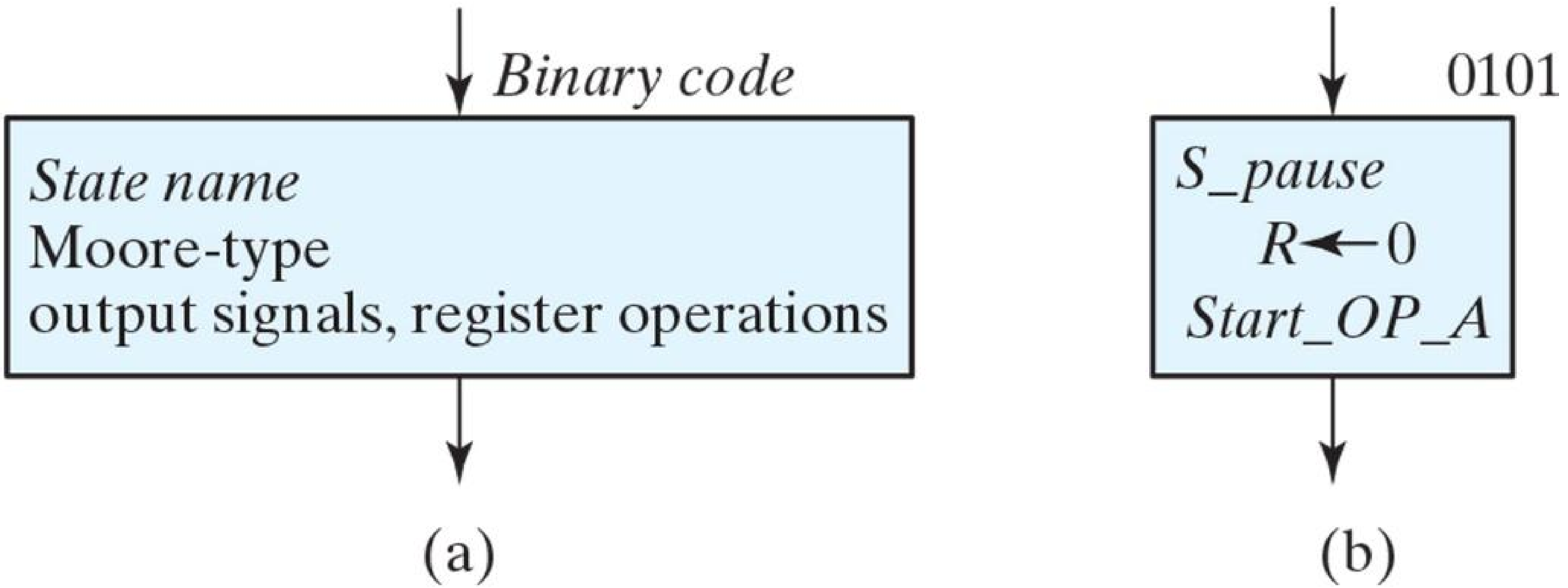


Figure 8.4
ASM chart decision box.

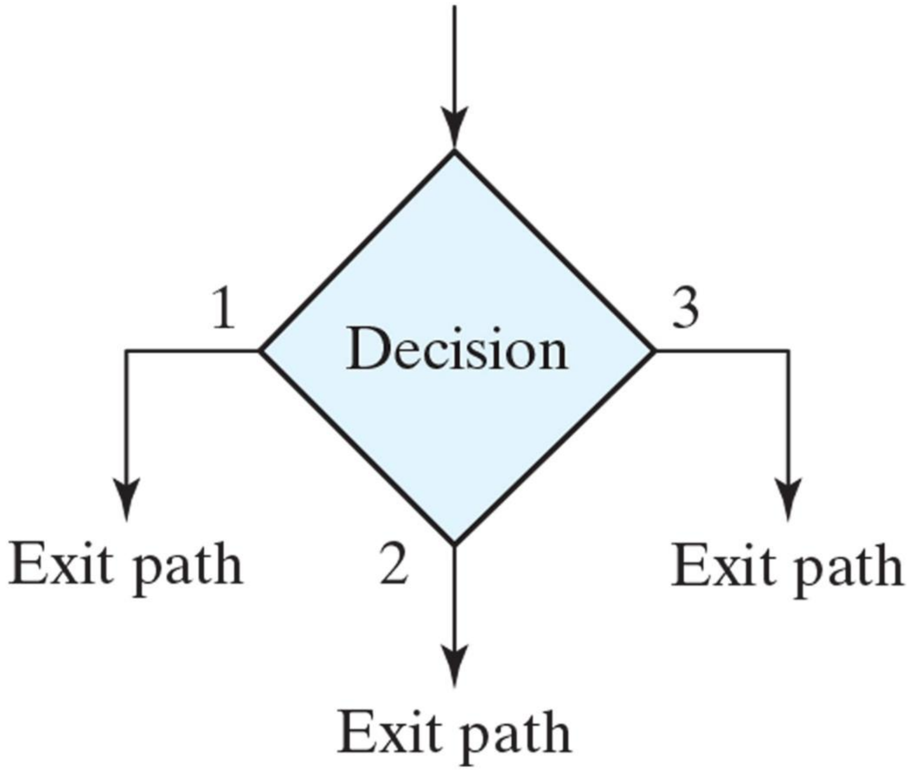
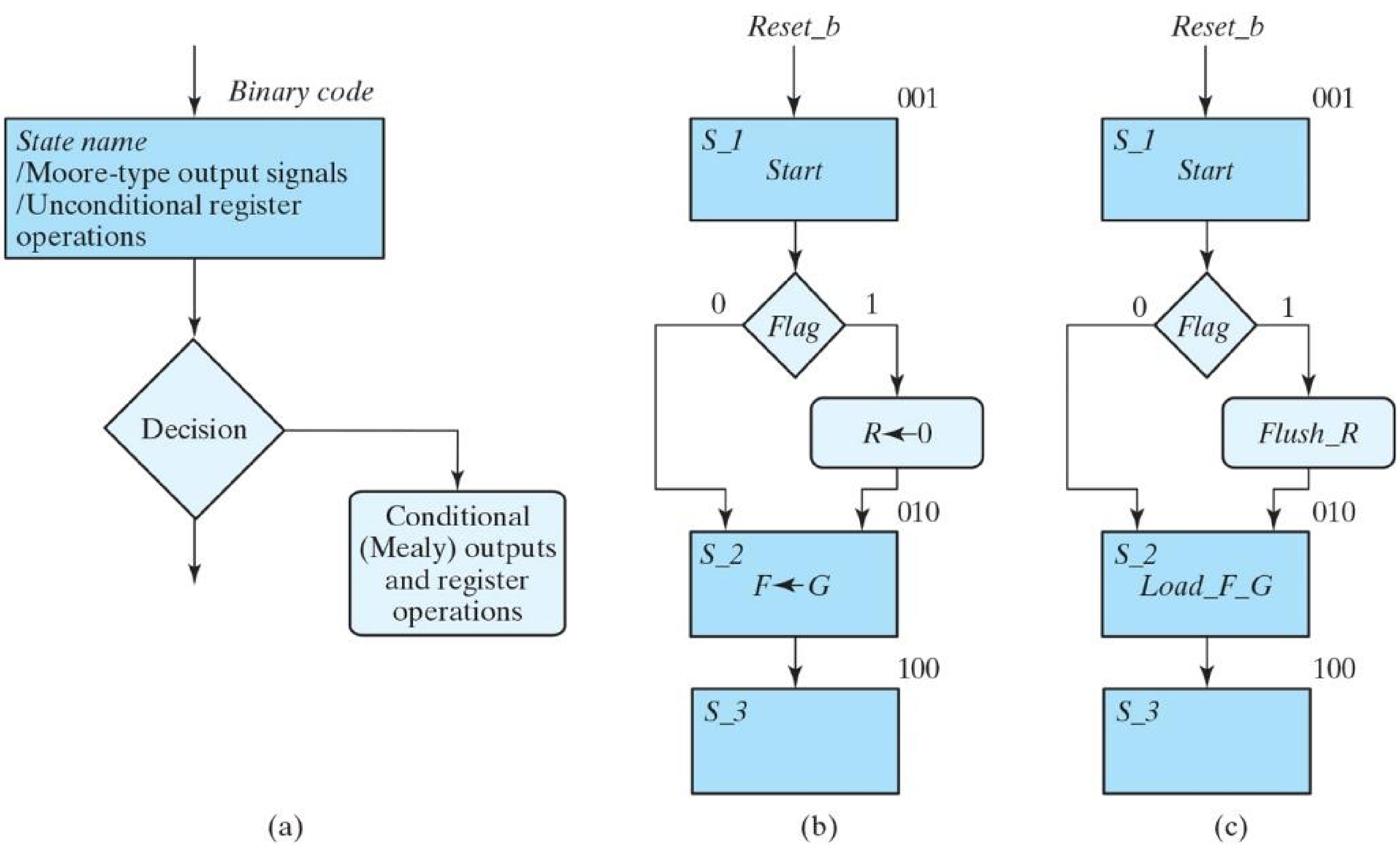
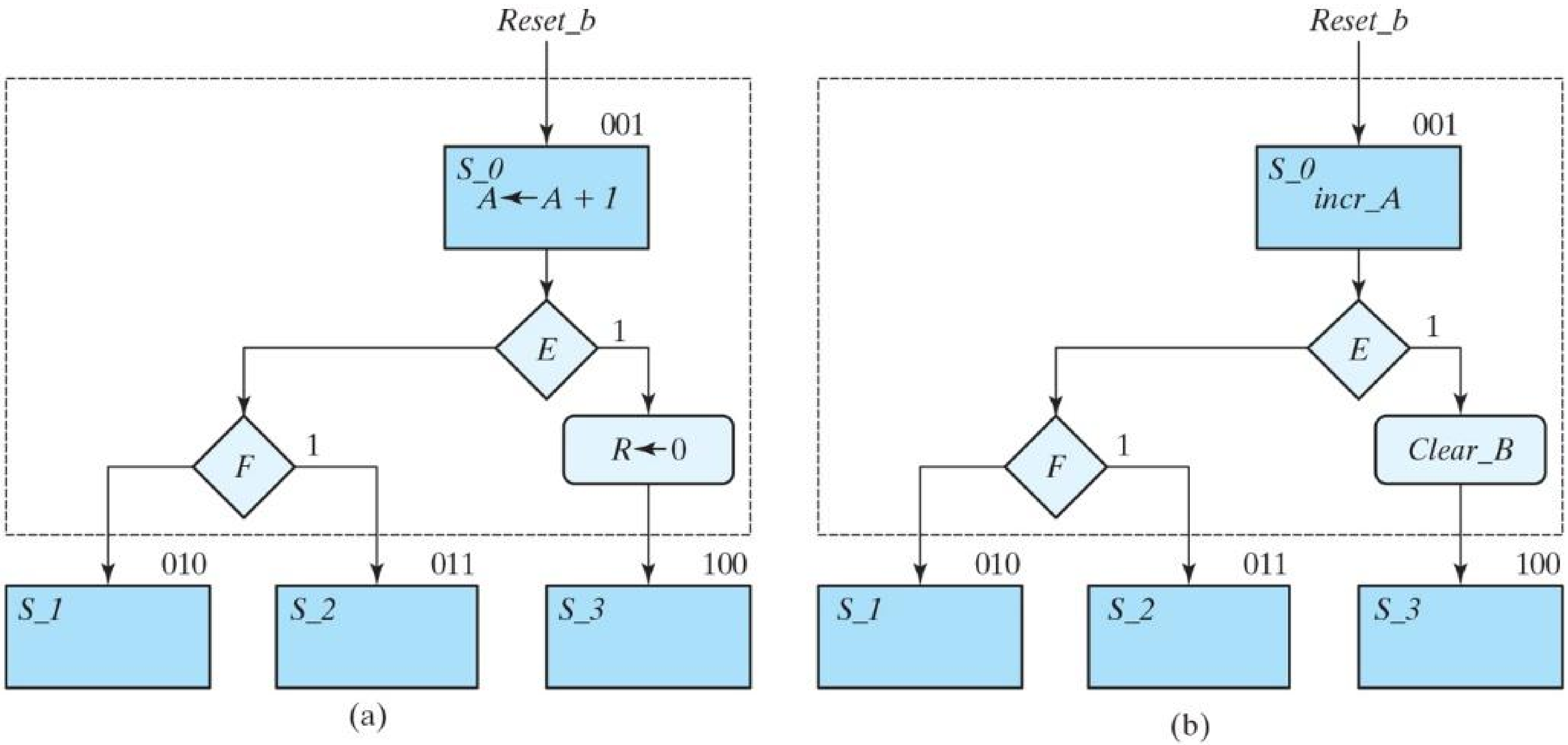


Figure 8.5
ASM chart conditional box and examples.



Pearson

Figure 8.6
ASM blocks.



Pearson

Figure 8.7
State diagram equivalent to the ASM chart of Fig. 8.6.

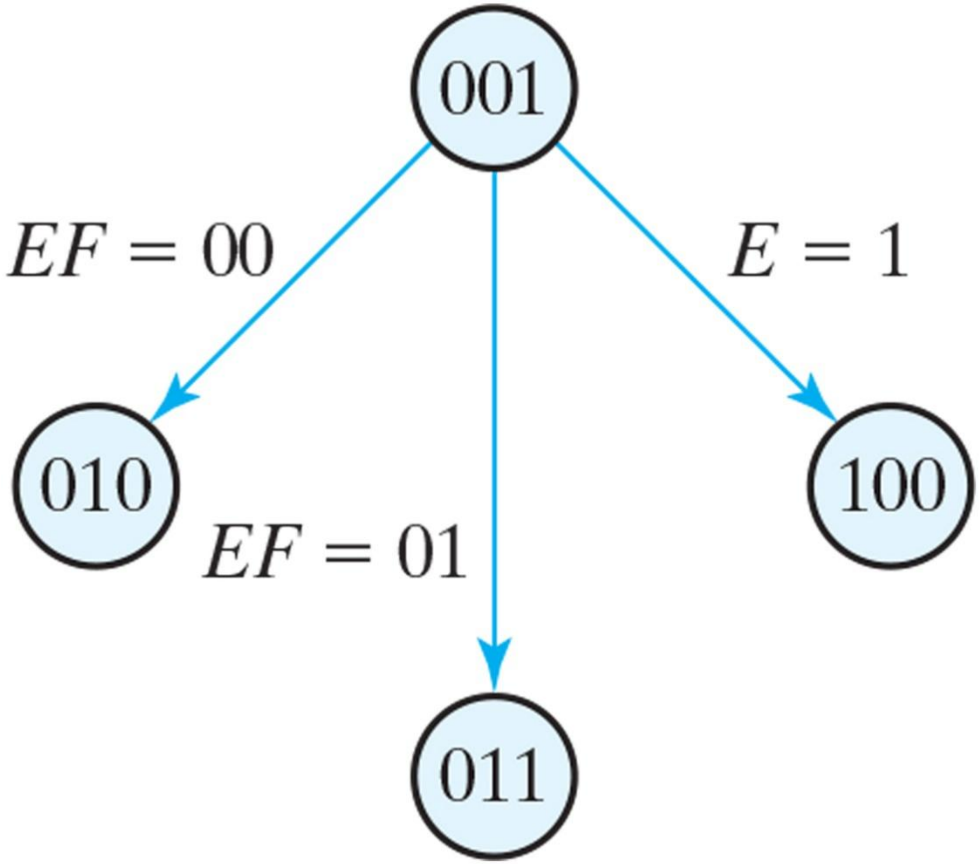
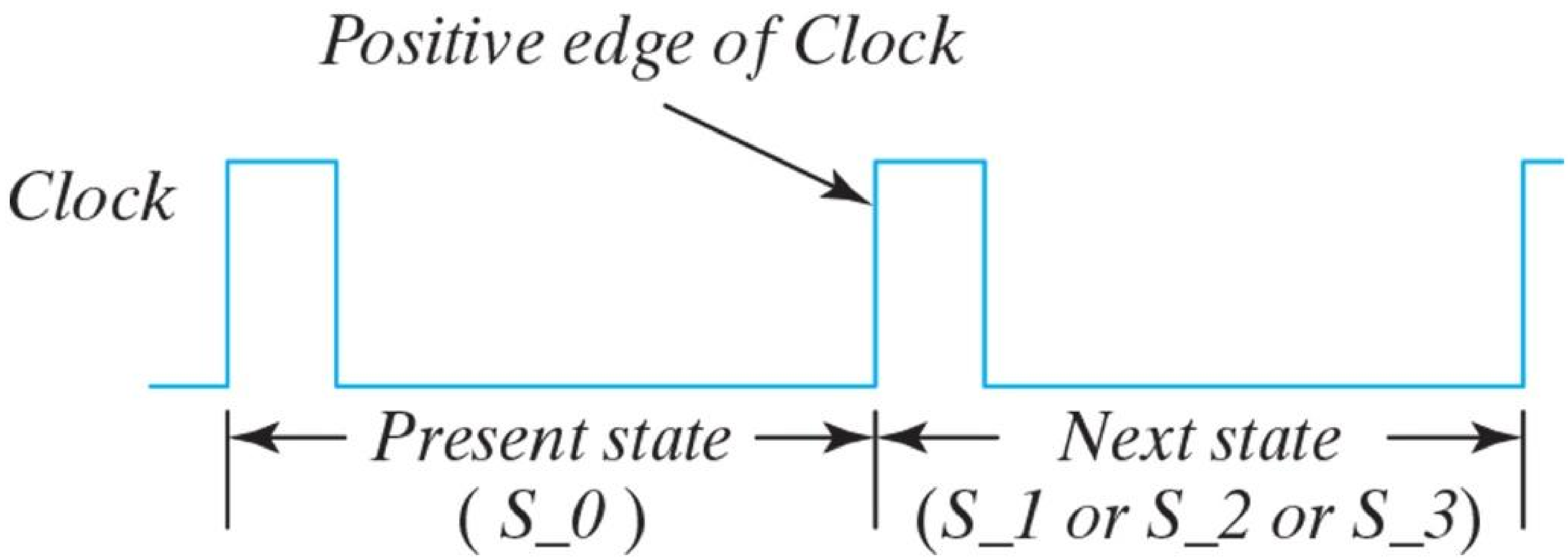


Figure 8.8
Transition between states.



Pearson

Figure PE 8.10

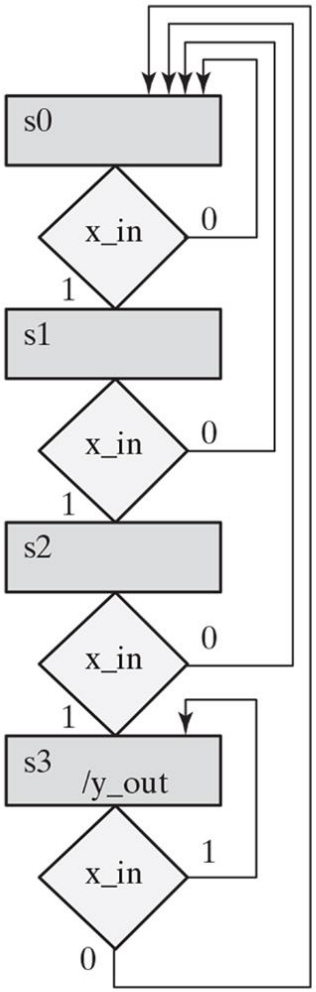


Figure 8.9
(a) Block diagram for design example; (b) ASM chart for controller state transitions, annotated with datapath register operations, asynchronous reset; (c) ASM chart for controller state transitions, annotated with datapath register operations, synchronous reset; (d) ASMD chart fora completely specified controller, identifying datapath operations and associated control signals,and asynchronous reset.

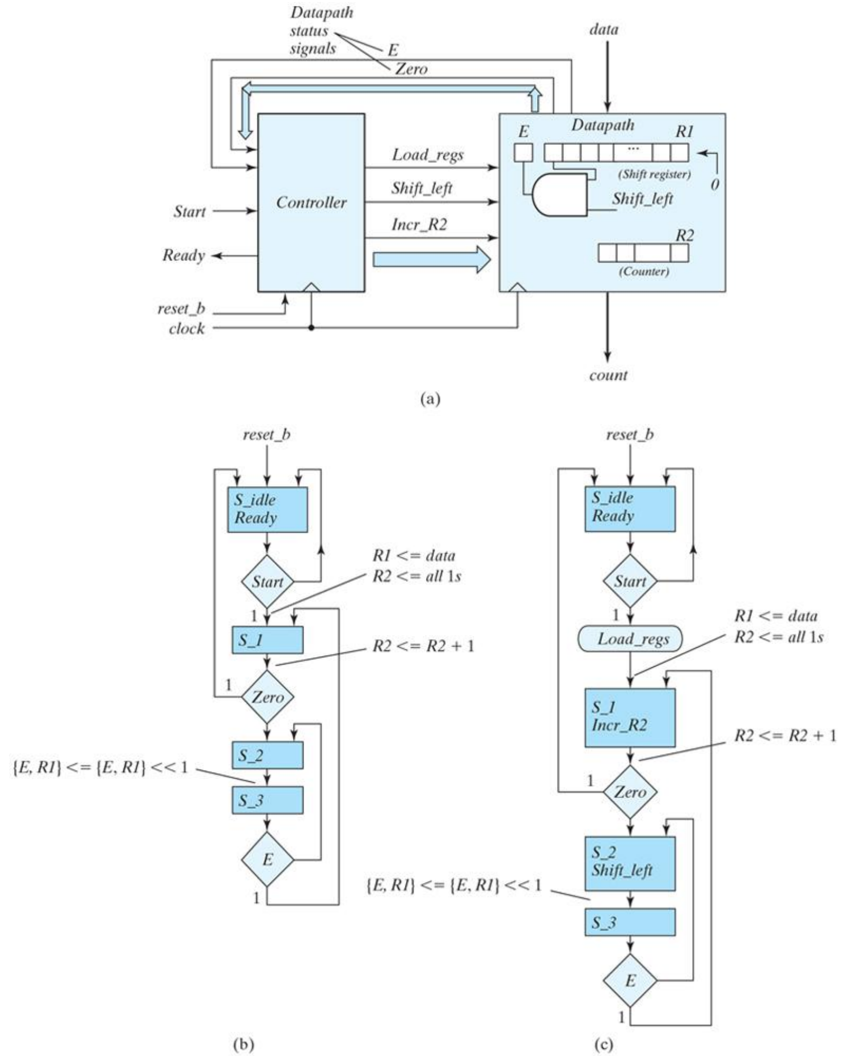


Table 8.3
Sequence of Operations for Design Example.

Counter				Flip-Flops			
A_3	A_2	A_1	A_0	E	F	Conditions	State
0	0	0	0	1	0	$A_2 = 0, A_3 = 0$	*S_1*
0	0	0	1	0	0		
0	0	1	0	0	0		
0	0	1	1	0	0		
0	1	0	0	0	0	$A_2 = 1, A_3 = 0$	
0	1	0	1	1	0		
0	1	1	0	1	0		
0	1	1	1	1	0		
1	0	0	0	1	0	$A_2 = 0, A_3 = 1$	
1	0	0	1	0	0		
1	0	1	0	0	0		
1	0	1	1	0	0		
1	1	0	0	0	0	$A_2 = 1, A_3 = 1$	
1	1	0	1	1	0		*S_2*
1	1	0	1	1	1		*S_idle*

Pearson

Figure 8.10
Datapath and controller for design example.

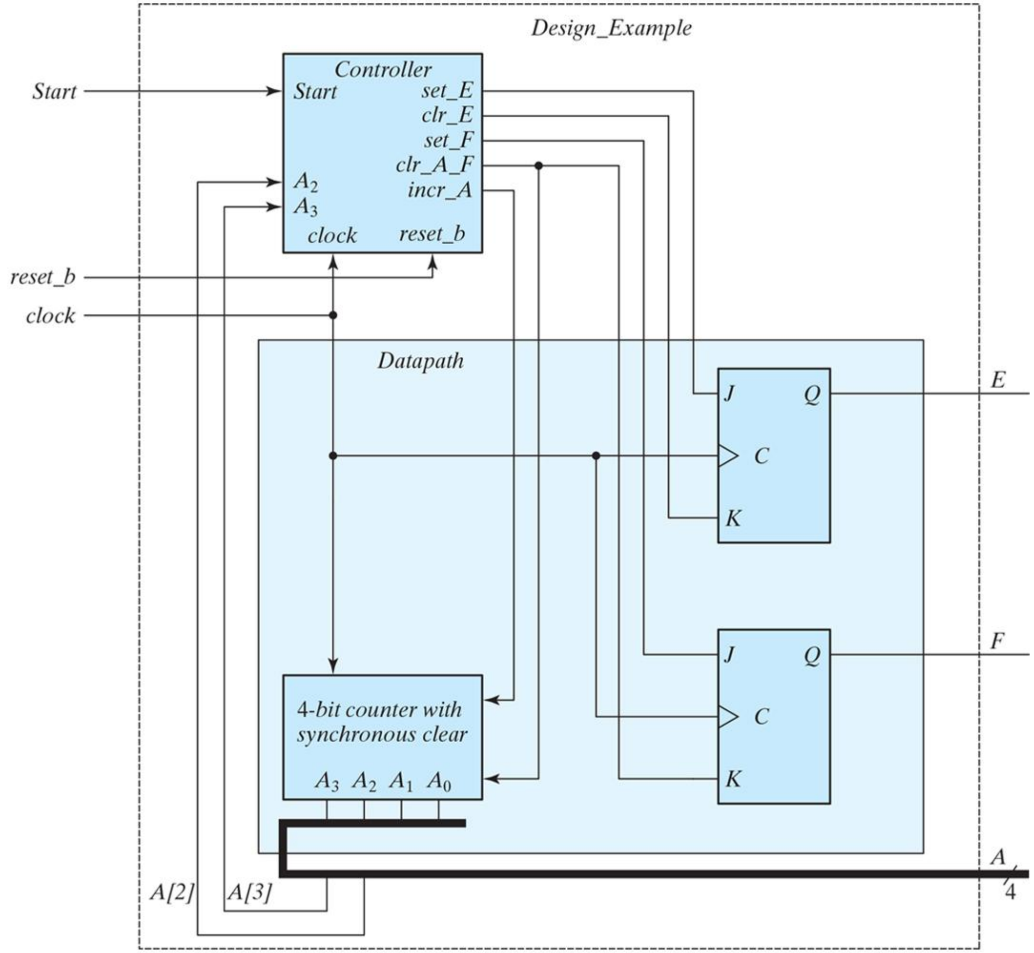
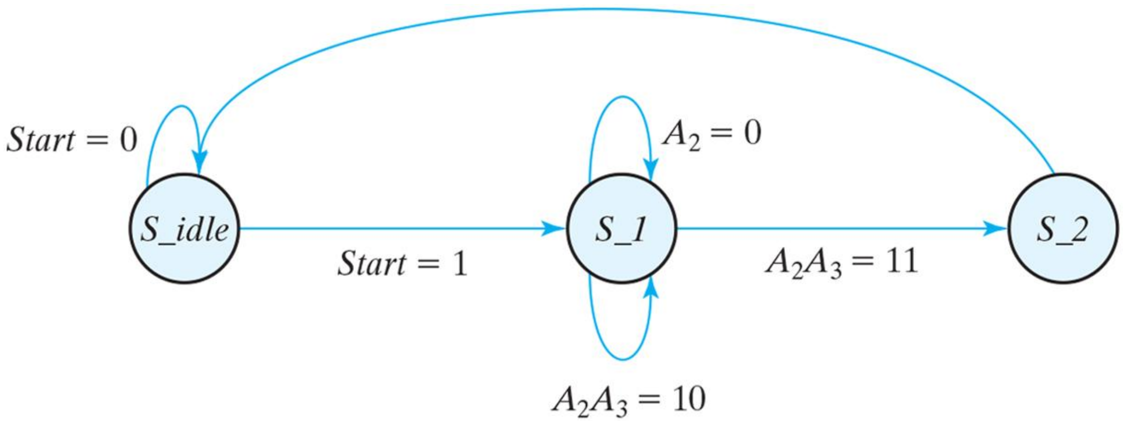


Figure 8.11
Register transfer-level description of design example.



(a)

$S\_idle \longrightarrow S\_1, clr\_A\_F$:		$A \longleftarrow 0, F \longleftarrow 0$
$S\_1 \longrightarrow S\_1, incr\_A$:		$A \longleftarrow A + 1$
	if $(A_2 = 1)$ then set_E:	$E \longleftarrow 1$
	if $(A_2 = 0)$ then clr_E:	$E \longleftarrow 0$
$S\_1 \longrightarrow S\_2, incr\_A$:		$F \longleftarrow 1$
$S\_2 \longrightarrow S\_idle$:		$A \longleftarrow A + 1$

(b)

Table 8.4
State Table for the Controller of Fig. 8.10.

Present-State Symbol	Present State		Inputs			Next State		Outputs				
	G_1	G_0	*Start*	A_2	A_3	G_1	G_0	*set_E*	*clr_E*	*set_F*	*clr_A_F*	*incr_A*
S_idle	0	0	0	X	X	0	0	0	0	0	0	0
S_idle	0	0	1	X	X	0	1	0	0	0	1	0
S_1	0	1	X	0	X	0	1	0	1	0	0	1
S_1	0	1	X	1	0	0	1	1	0	0	0	1
S_1	0	1	X	1	1	1	1	1	0	0	0	1
S_2	1	1	X	X	X	0	0	0	0	1	0	0

Pearson

Figure 8.12
Logic diagram of the control unit for Fig. 8.10.

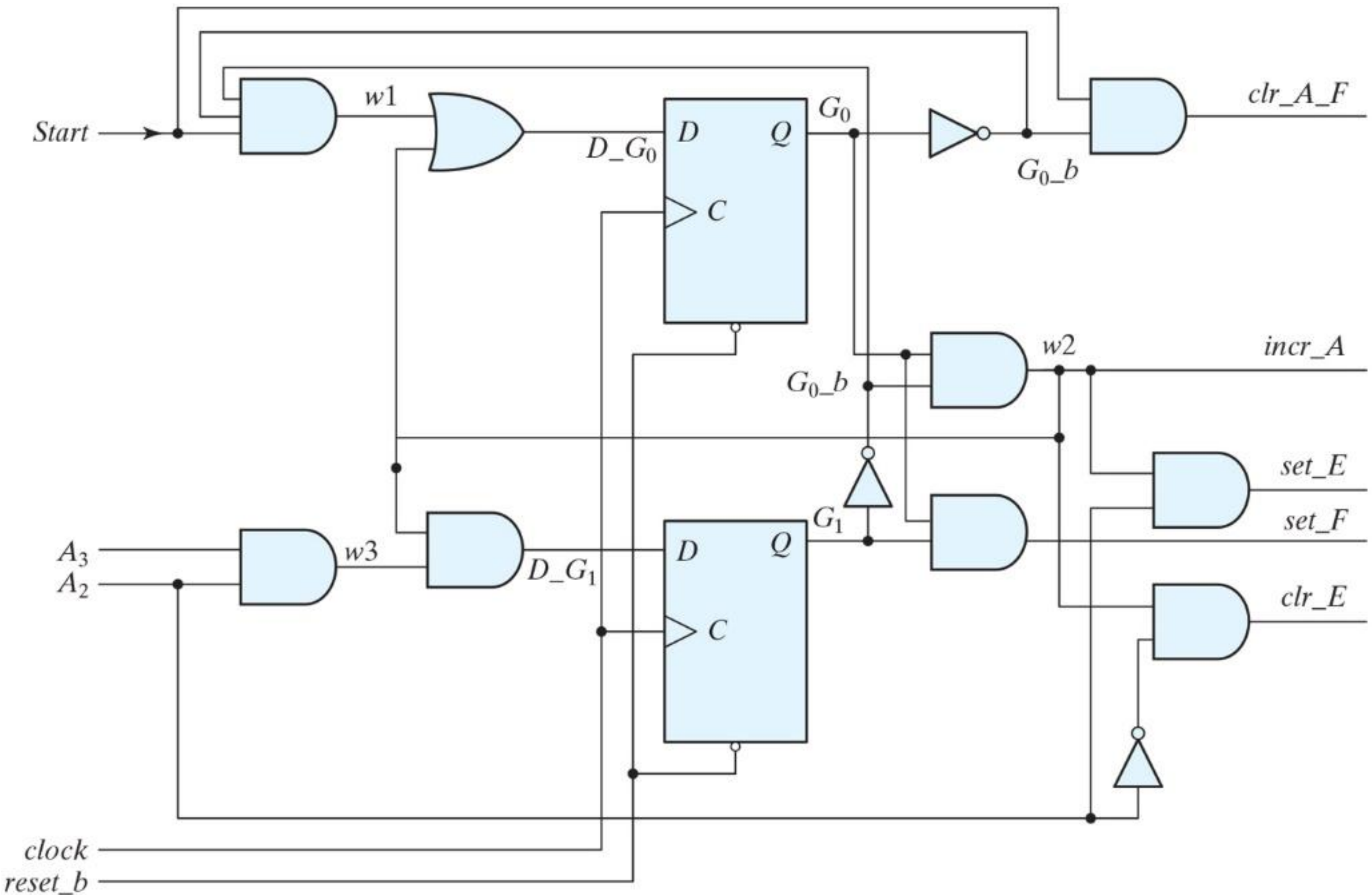


Figure 8.13
Simulation results for *Design_Example_RTL*.

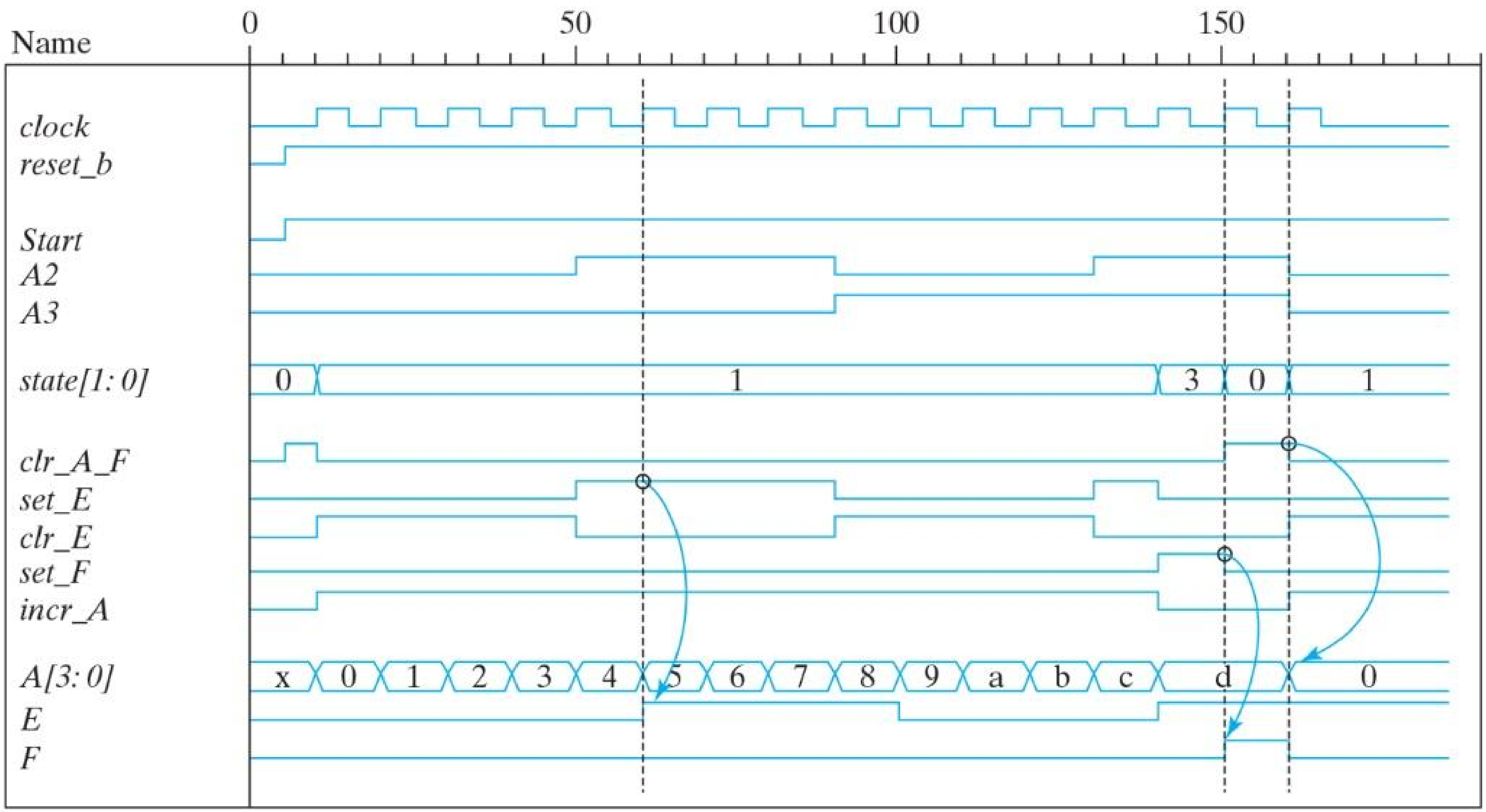


Figure 8.14
(a) Block diagram and (b) datapath of a binary multiplier.

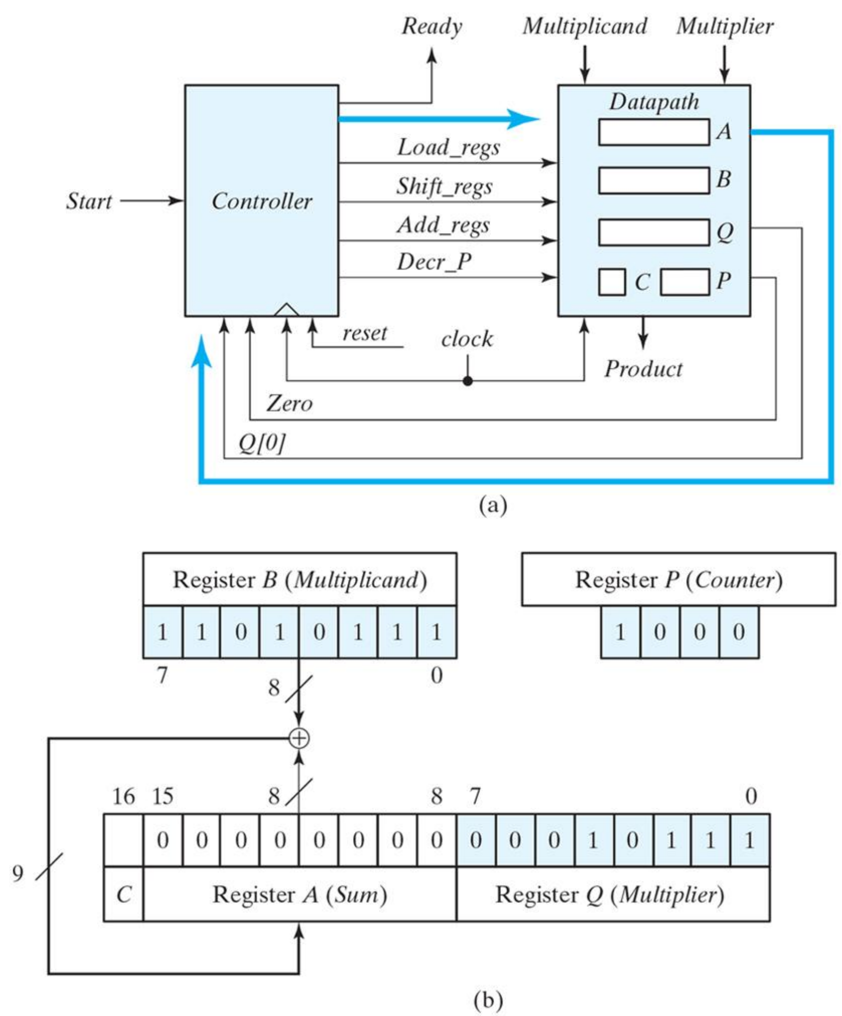
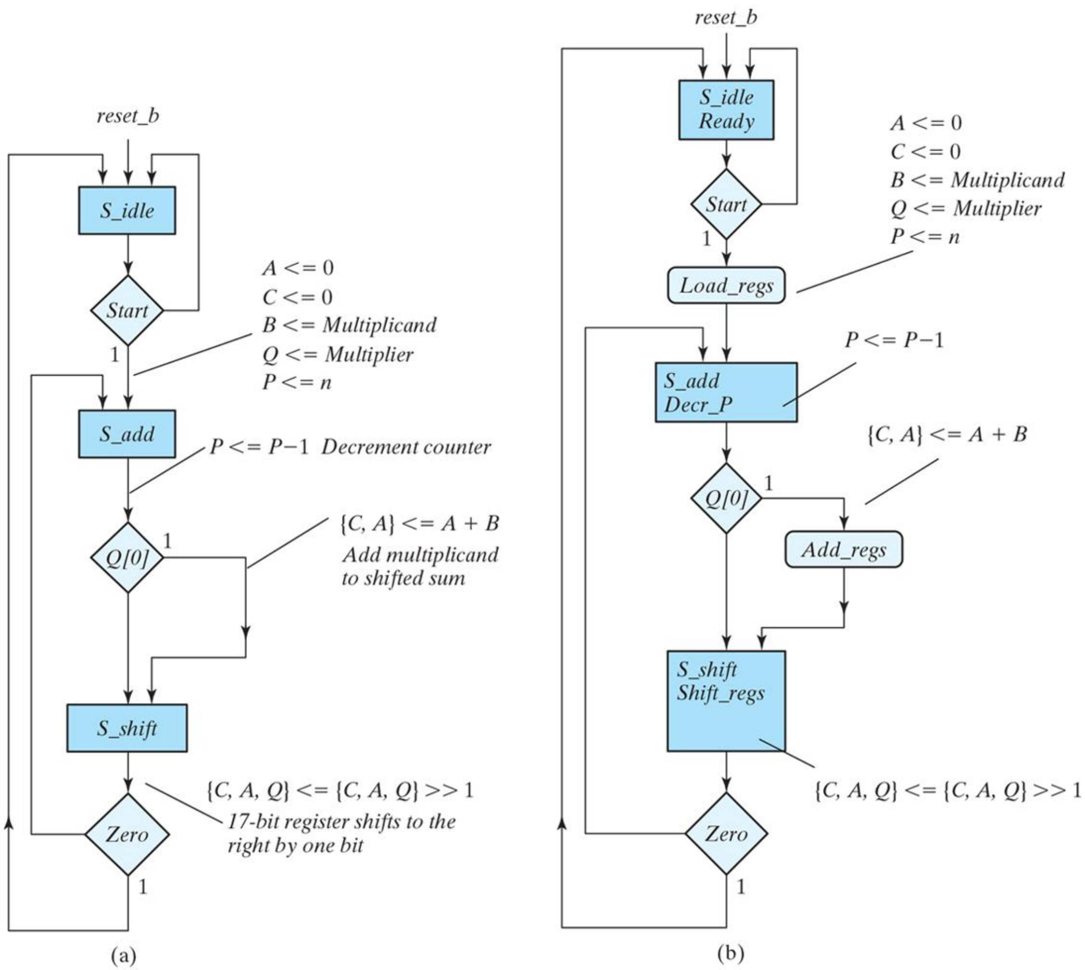


Figure 8.15
ASMD chart for binary multiplier.

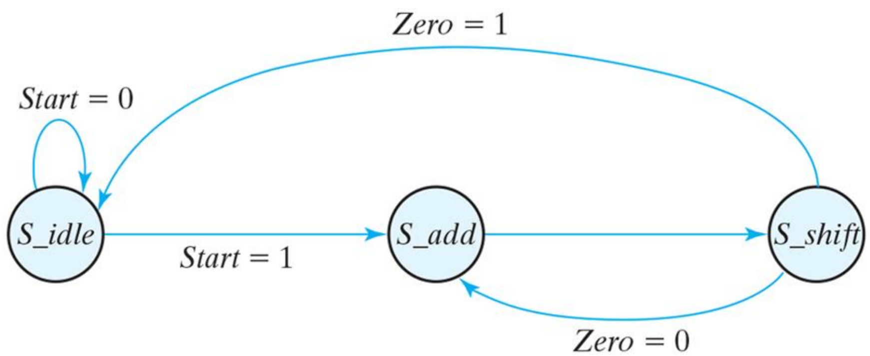


Pearson

Table 8.5
Numerical Example For Binary Multiplier.

Multiplicand $B = 10111_2 = 17_H = 23_{10}$ Multiplier $Q = 10011_2 = 13_H = 19_{10}$	C	A	Q	P
Multiplier in Q	0	00000	10011	101
$Q_0 = 1$; add B		$\underline{10111}$		
First partial product	0	10111		100
Shift right CAQ	0	01011	11001	
$Q_0 = 1$; add B		$\underline{10111}$		
Second partial product	1	00010		011
Shift right CAQ	0	10001	01100	
$Q_0 = 0$; shift right CAQ	0	01000	10110	010
$Q_0 = 0$; shift right CAQ	0	00100	01011	001
$Q_0 = 1$; add B		$\underline{10111}$		
Fifth partial product	0	11011		
Shift right CAQ	0	01101	10101	000
Final product in $AQ = 0110110101_2 = 1b5_H$				

Figure 8.16
Control specifications for binary multiplier.



(a)

State Transition		Register Operations
From	To	
S_idle		*Initial state*
S_idle	*S_add*	$A <= 0, C <= 0, P <= dp\_width$ $A <= Multiplicand, Q <= Multiplier$
S_add	*S_shift*	$P <= P - 1$ $if\ (Q[0])\ then\ (A <= A + B, C <= C_{out})$
S_shift		$shift\ right\ \{CAQ\}, C <= 0$

(b)

Table 8.6
State Assignment for Control.

State	Binary	Gray Code	One-Hot
S_idle	00	00	001
S_add	01	01	010
S_shift	10	11	100

Table 8.7
State Table for Control Circuit.

Present-State Symbol	Present State		Inputs			Next State		Outputs				
	G_1	G_0	*Start*	*Q[0]*	*Zero*	G_1	G_0	*Ready*	*Load_regs*	*Decr_P*	*Add_regs*	*Shift_regs*
S_idle	0	0	0	X	X	0	0	1	0	0	0	0
S_idle	0	0	1	X	X	0	1	1	1	0	0	0
S_add	0	1	X	0	X	1	0	0	0	1	0	0
S_add	0	1	X	1	X	1	0	0	0	1	1	0
S_shift	1	0	X	X	0	0	1	0	0	0	0	1
S_shift	1	0	X	X	1	0	0	0	0	0	0	1

Pearson

Figure 8.17

Logic diagram derived from Table 8.7 for controlling a binary multiplier using a sequence register and decoder.

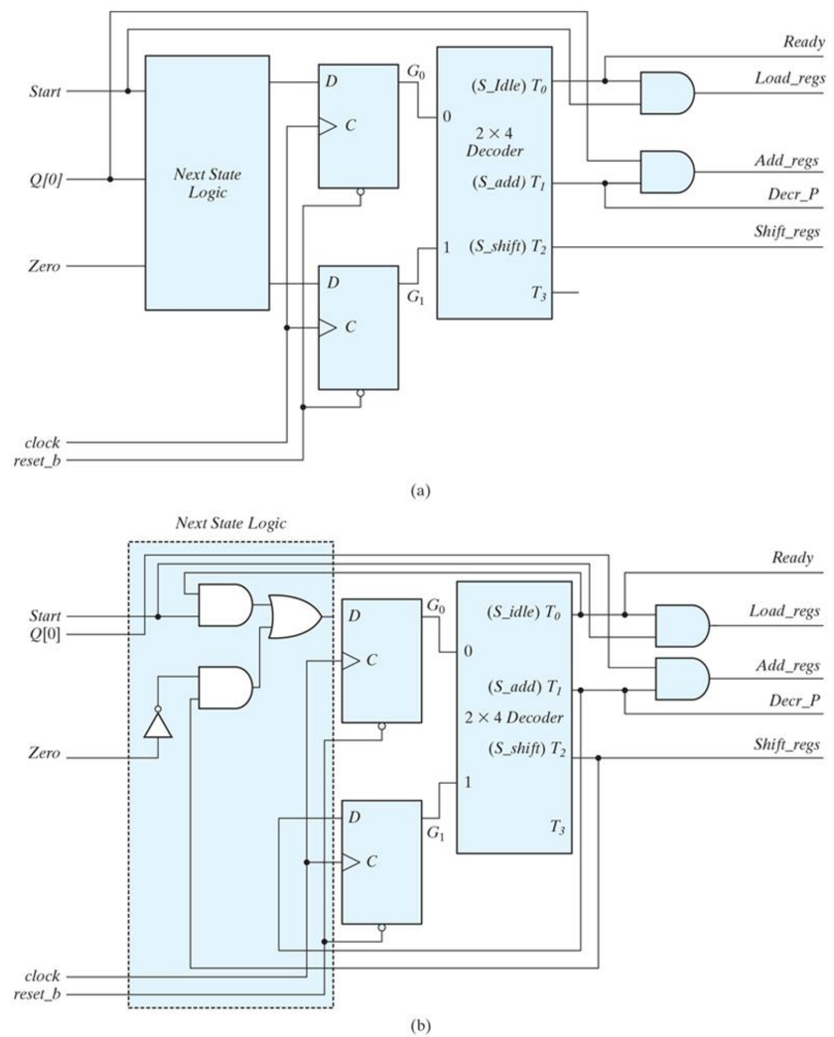


Figure 8.18
Logic diagram for one-hot state controller.

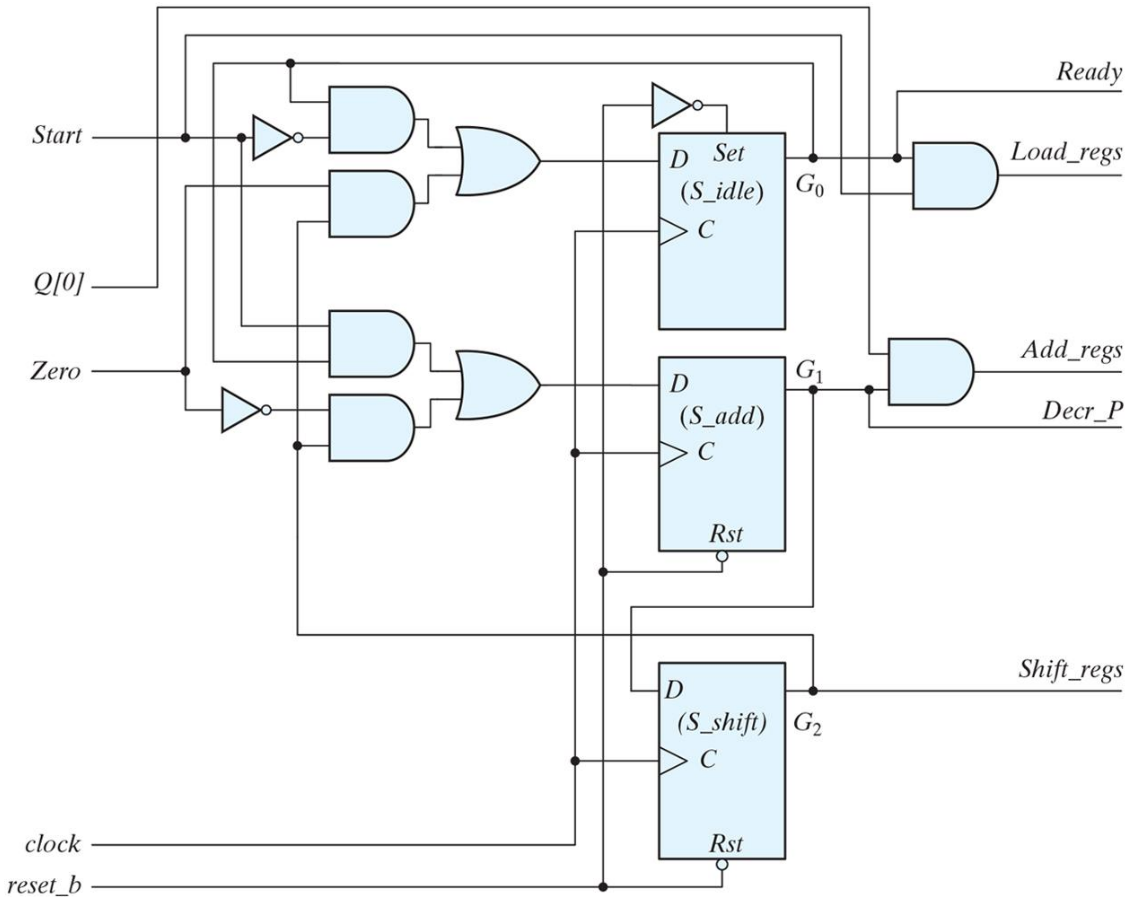


Figure 8.19
Simulation waveforms for one-hot state controller.

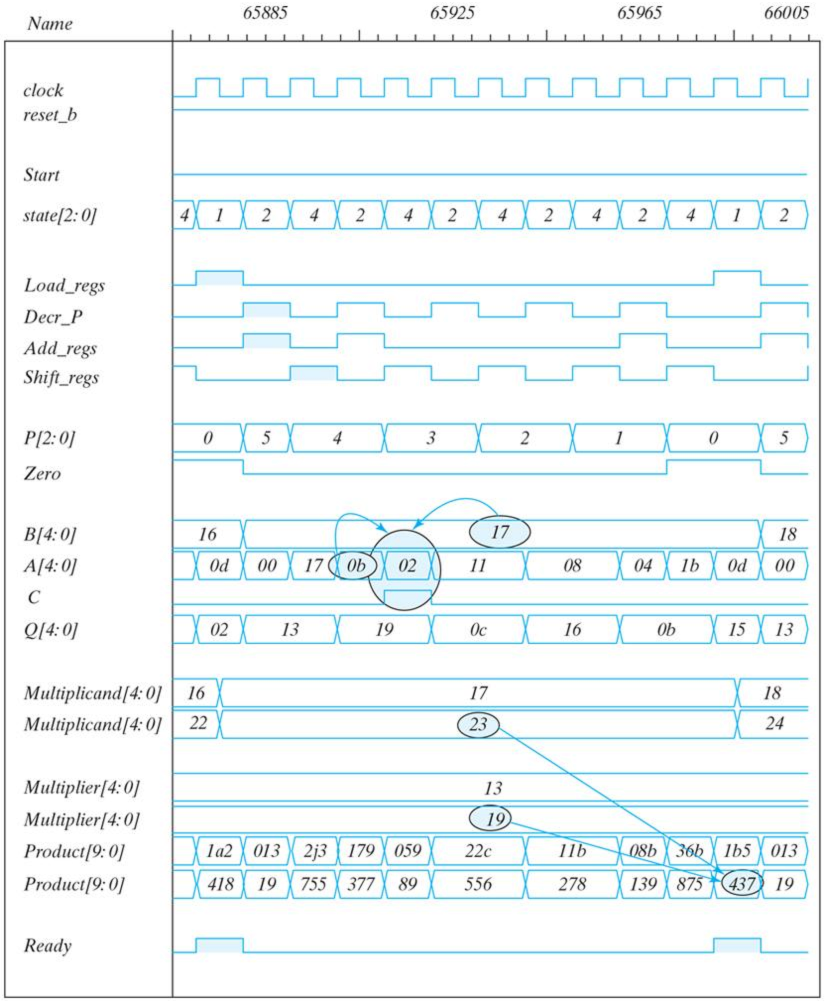


Figure 8.20
Example of ASM chart with four control inputs.

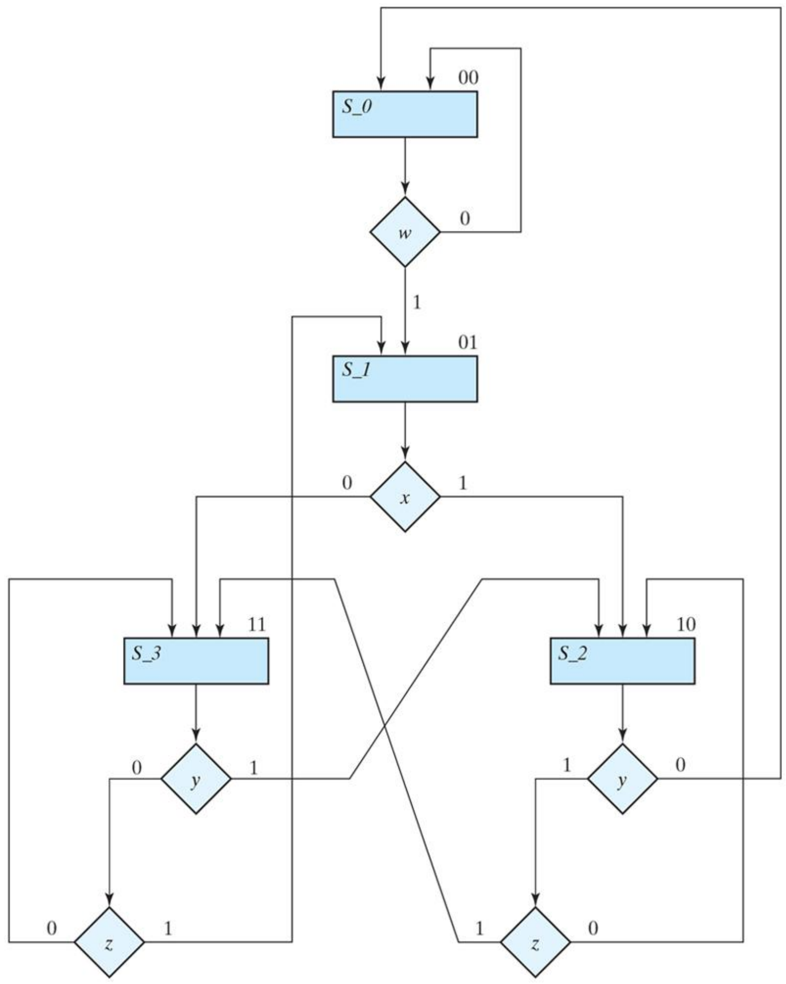


Figure 8.21
Control implementation with multiplexers.

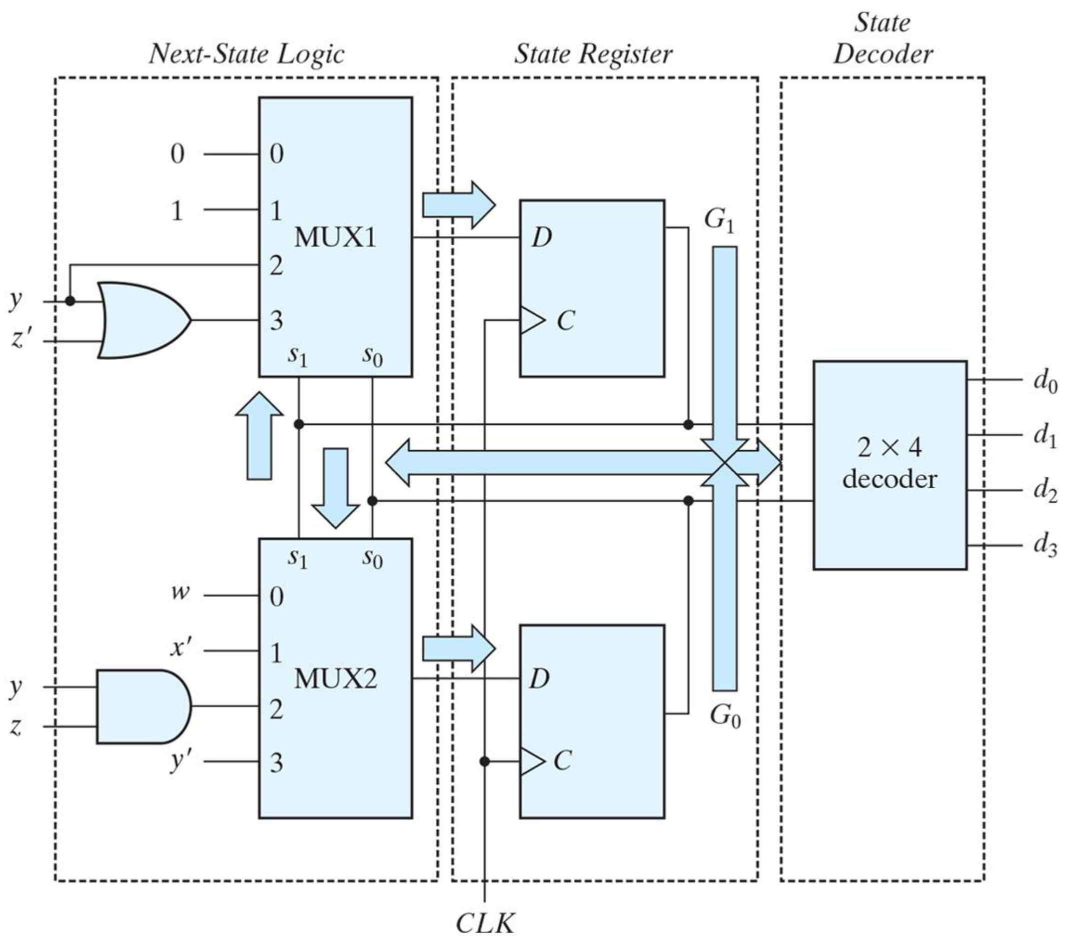


Table 8.8
Multiplexer Input Conditions.

Present State		Next State		Input Condition	Inputs	
G_1	G_0	G_1	G_0	s	MUX1	MUX2
0	0	0	0	w'		
0	0	0	1	w	0	w
0	1	1	0	x		
0	1	1	1	x'	1	x'
1	0	0	0	y'		
1	0	1	0	yz'		
1	0	1	1	yz	$yz' + yz = y$	yz
1	1	0	1	$y'z$		
1	1	1	0	y		
1	1	1	1	$y'z'$	$y + y'z' = y + z'$	$y'z + y'z' = y'$

Pearson

Figure 8.22
Block diagram and ASMD chart for count-of-ones circuit.

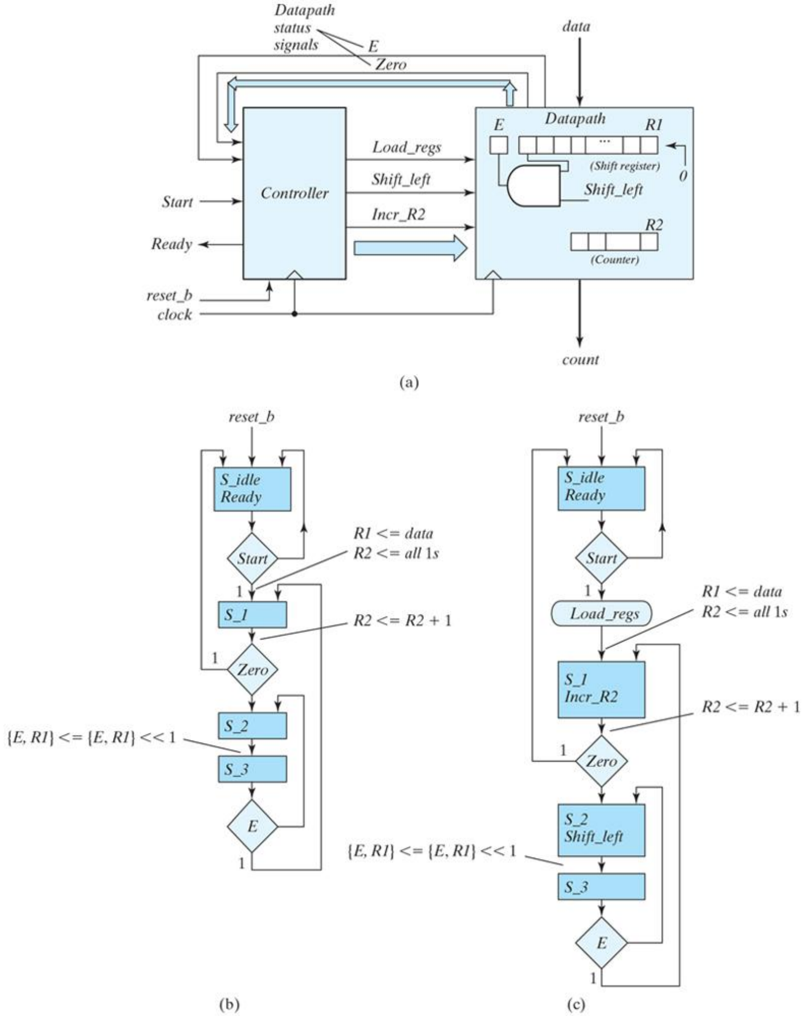
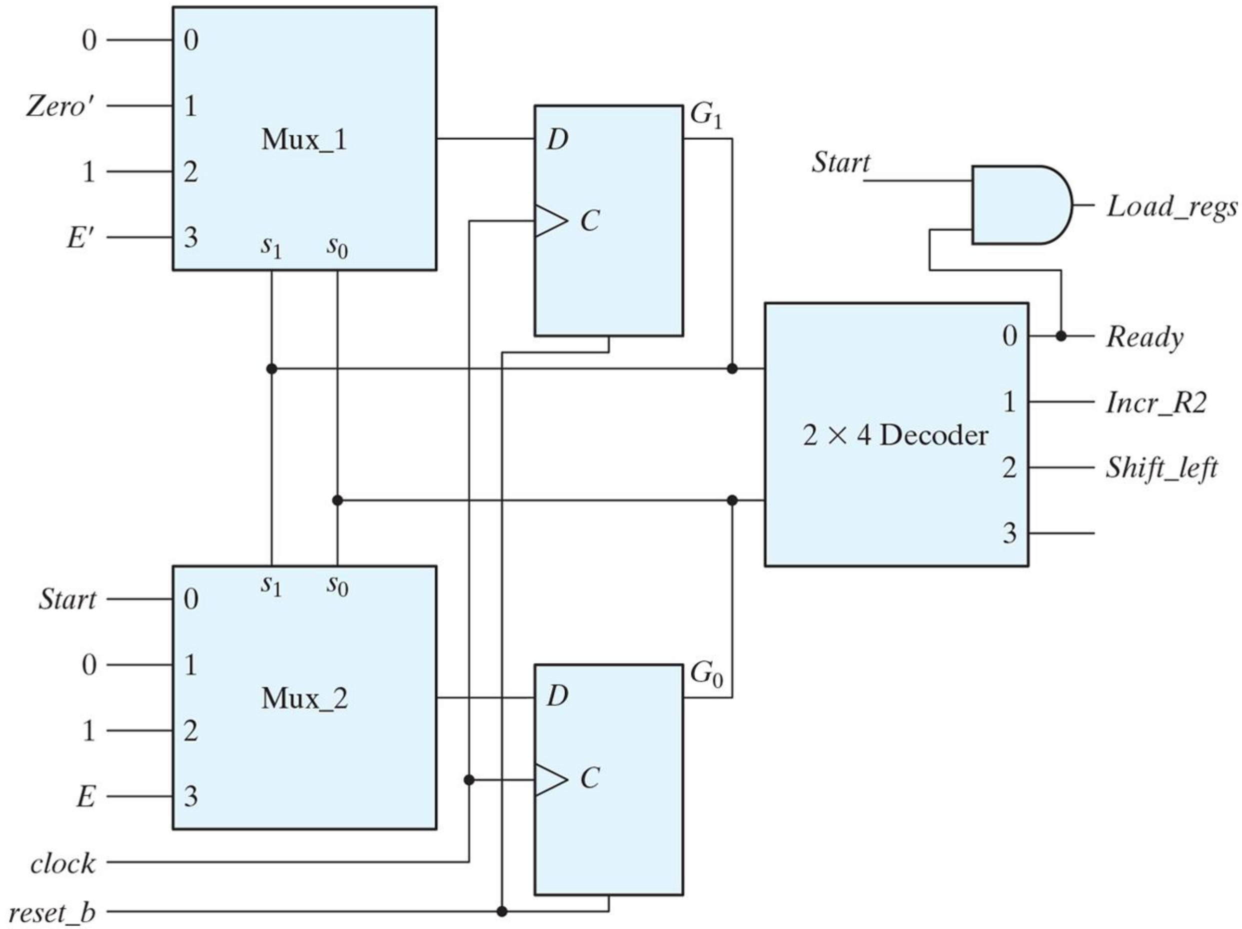


Figure 8.23
Control implementation for count-of-ones circuit.

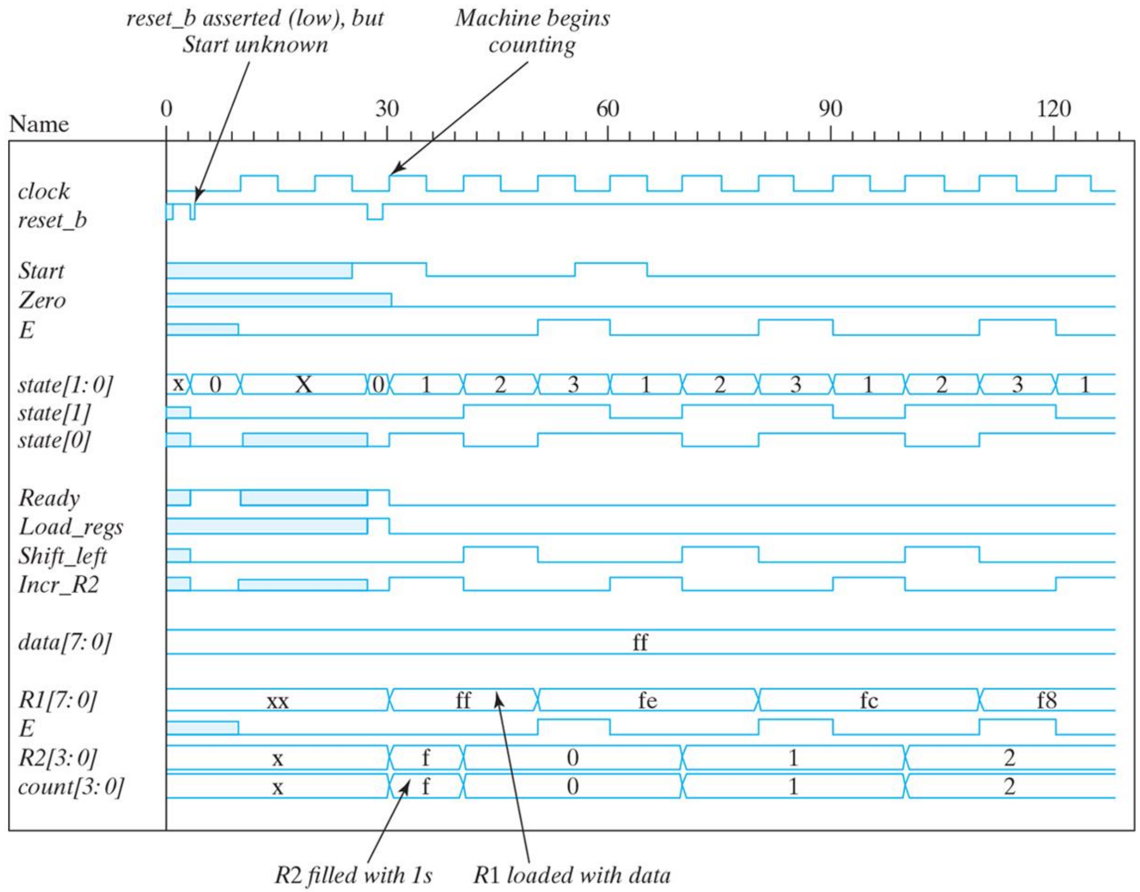


Pearson

Table 8.9
Multiplexer Input Conditions for Design Example.

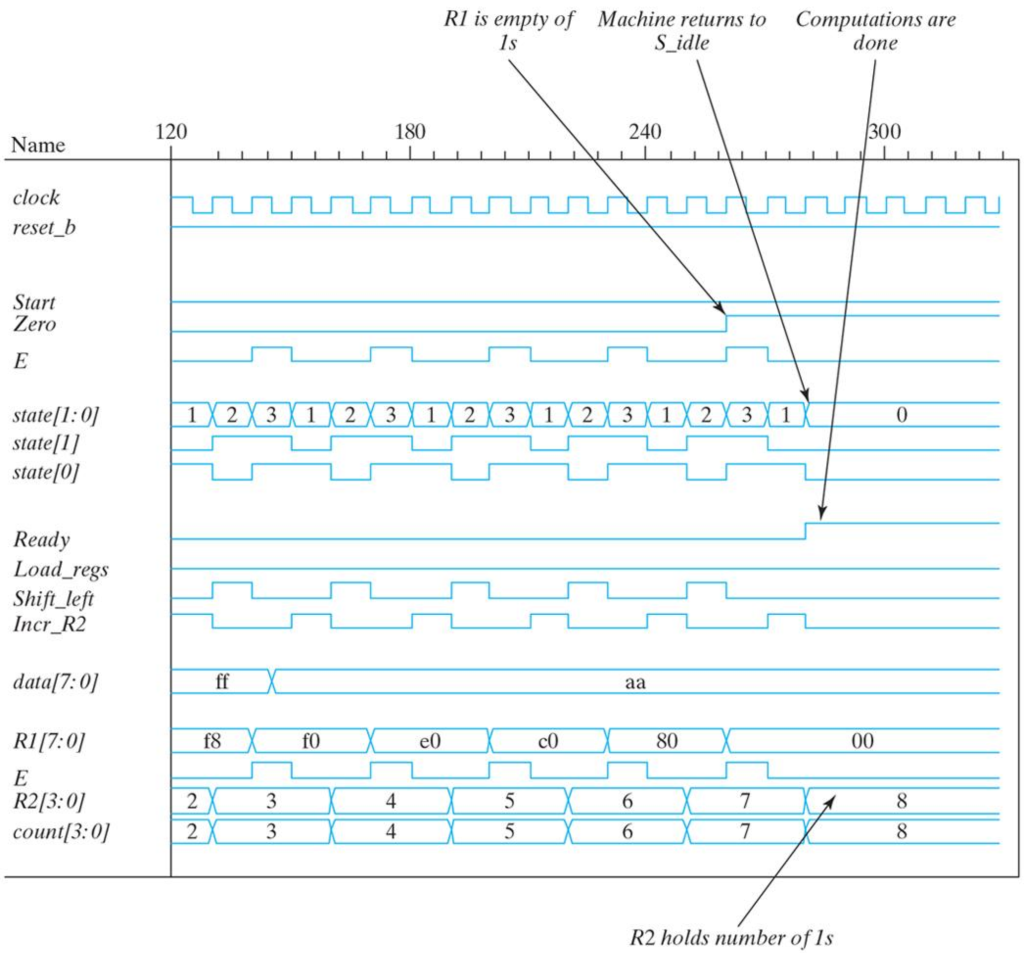
Present State		Next State		Input Conditions	Multiplexer Inputs	
G_1	G_0	G_1	G_0		MUX1	MUX2
0	0	0	0	*Start'*		
0	0	0	1	*Start*	0	*Start*
0	1	0	0	*Zero*		
0	1	1	0	*Zero'*	*Zero'*	0
1	0	1	1	*None*	1	1
1	1	1	0	*E'*		
1	1	0	1	*E*	*E'*	*E*

Figure 8.24
Simulation waveforms for count-of-ones circuit.



(a)

Figure 8.24
Simulation waveforms for count-of-ones circuit.



(b)

Figure P8.10
Control state diagram for Problems 8.10 and 8.11.

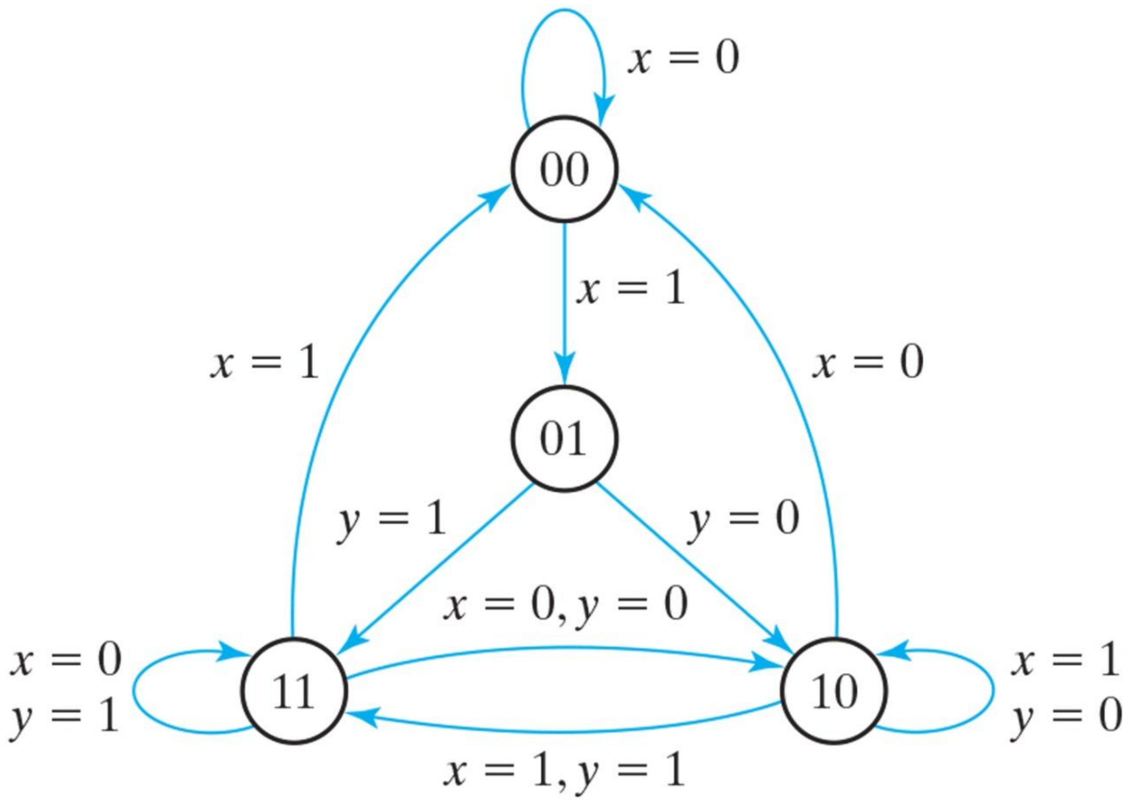
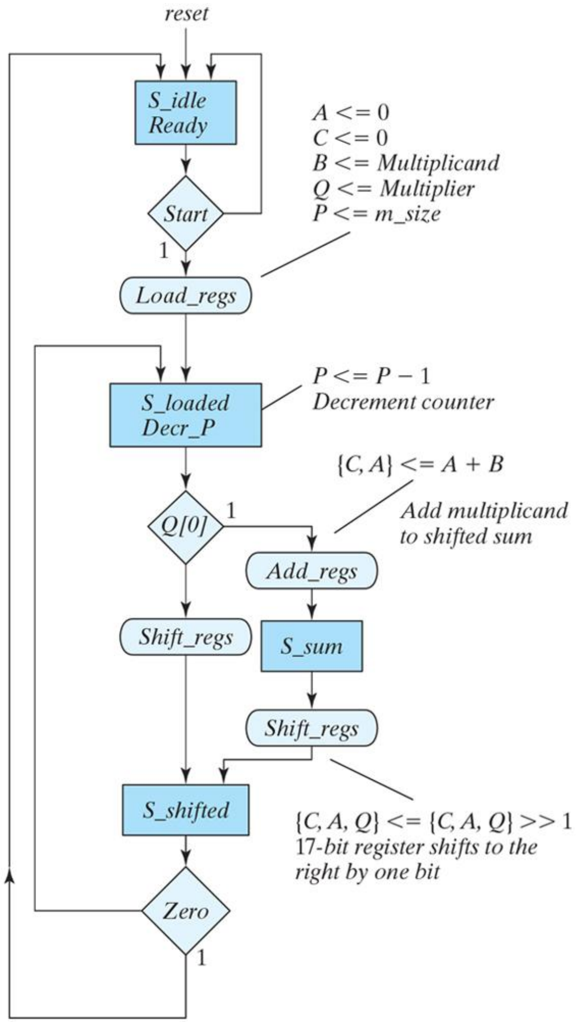


Figure P8.22
ASMD chart for Problem 8.22.



Pearson

Figure P8.23
ASMD chart for Problem 8.23.

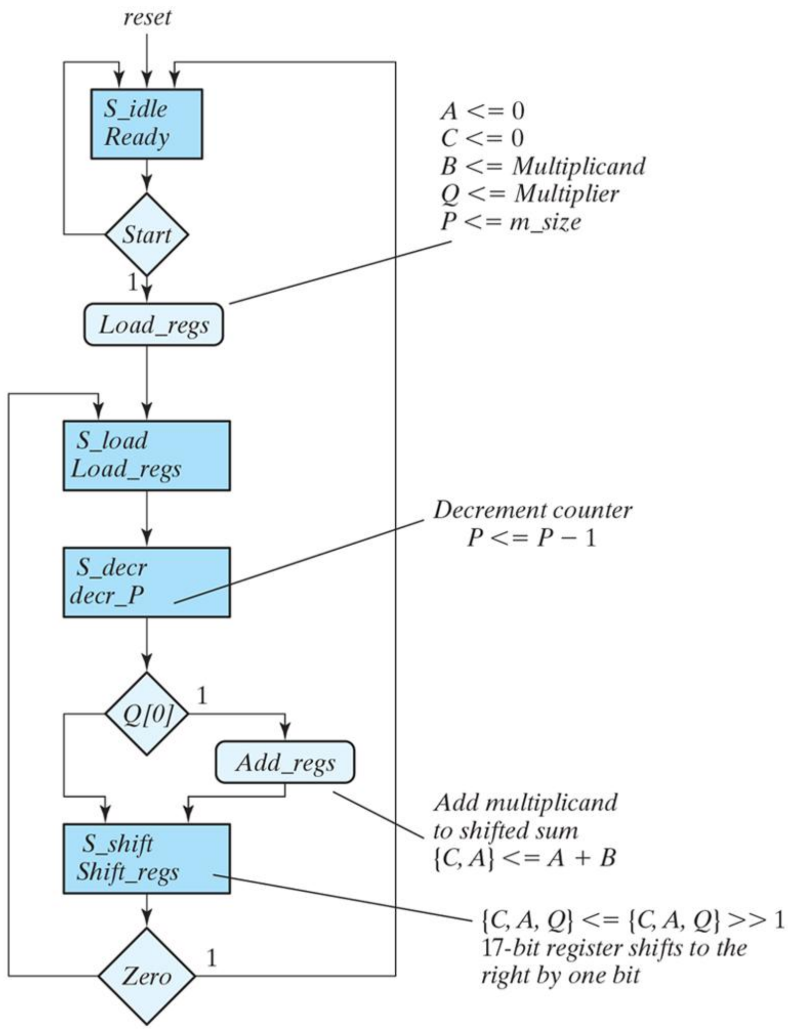


Figure P8.29
ASMD chart for Problem 8.29.

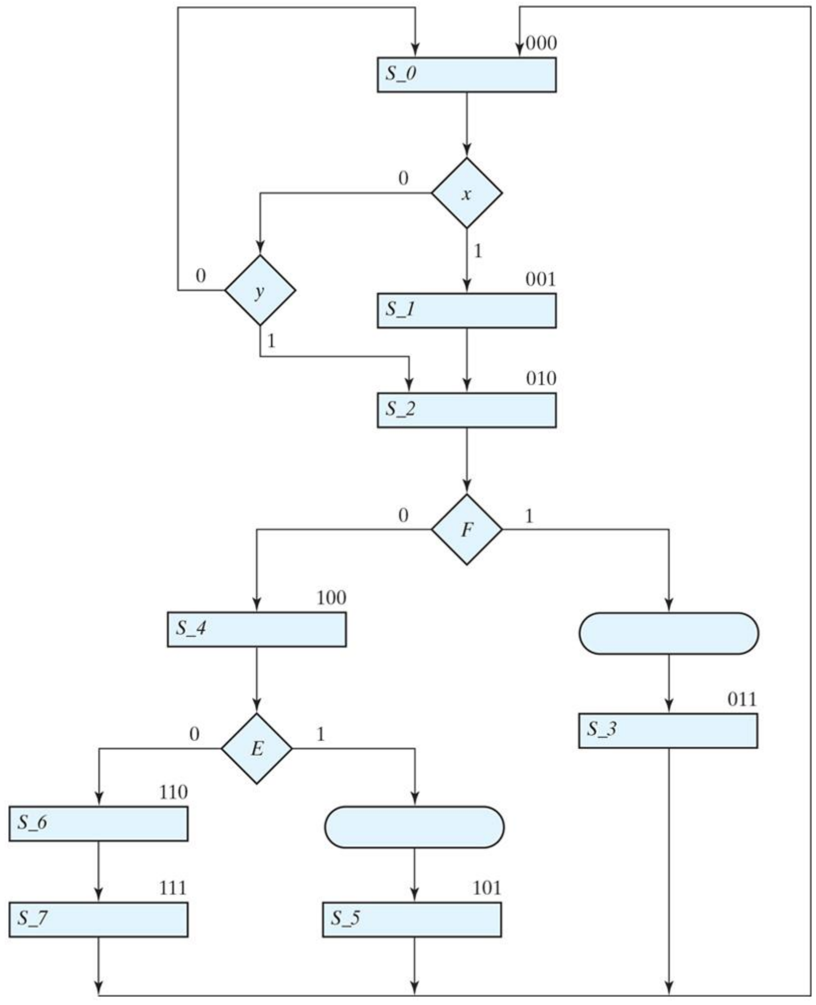


Figure P8.37
(a) Alternative circuit for a ones counter.
(b) ASMD Chart for Problem 8.37.

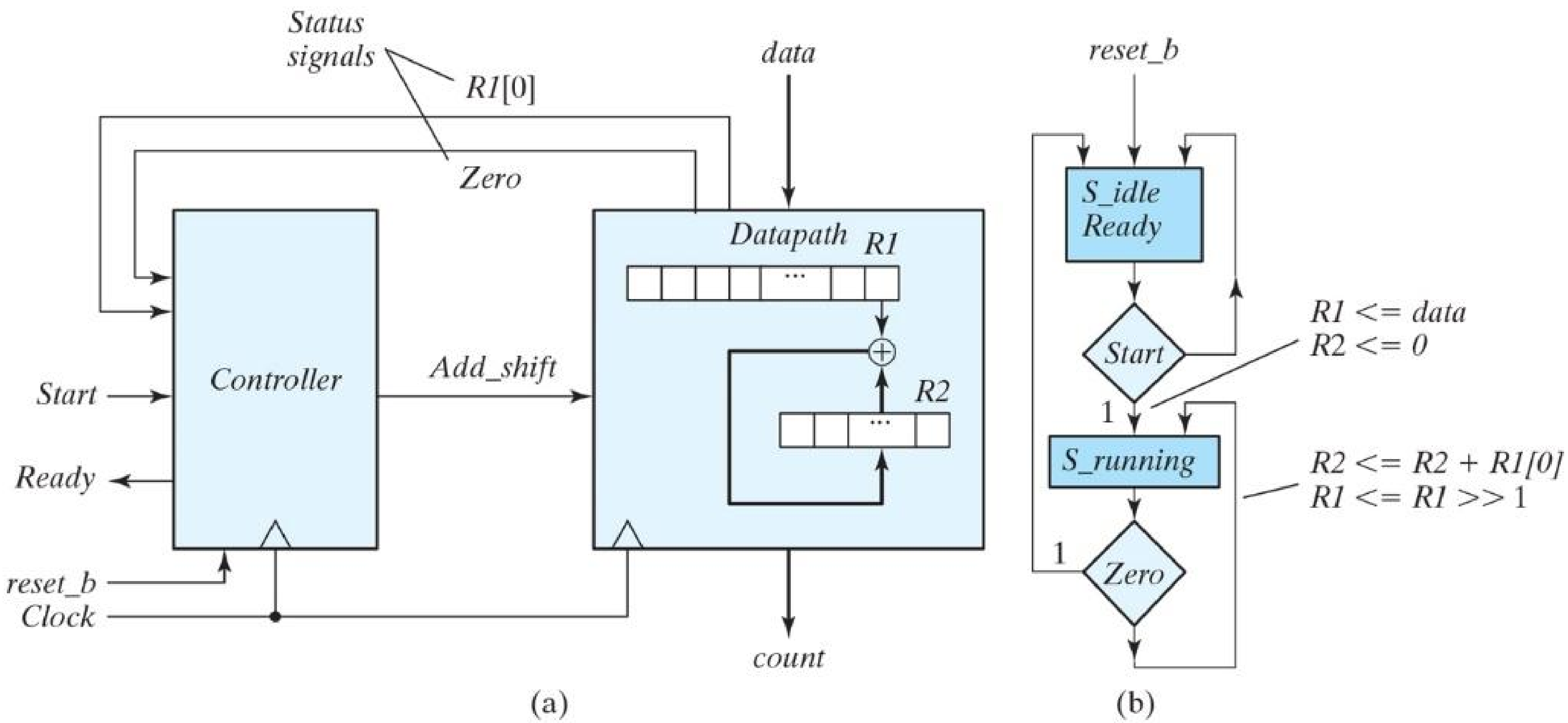


Figure P8.41
Two-stage pipeline register: Datapath unit and ASMD chart.